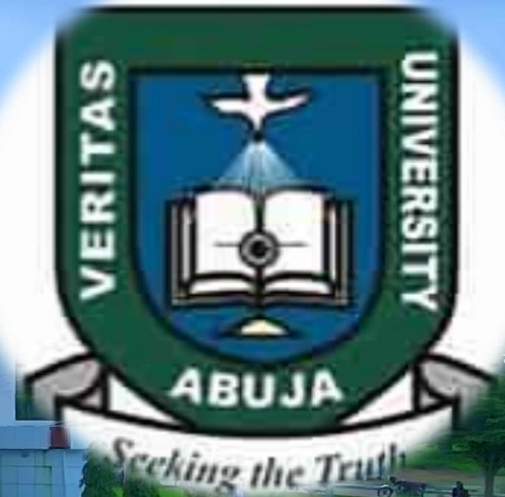


SEN201 LECTURE NOTE



**INTRODUCTION TO
SOFTWARE ENGINEERING**

2 CREDIT UNITS

DR ANAYO C.I
(Analyst Analyst)

ikegwua@veritas.edu.ng
OCT. 2025

COURSE OUTLINE

- Software engineering concepts and principles; design, development, and testing of software systems
 - Introduction to the software life cycle
 - Requirements, design, and testing
 - Review of principles of object orientation
 - Object-oriented analysis using UML.
- Frameworks and APIs
 - Introduction to a client-server architecture
 - Analysis, design, and programming of team projects including user interface considerations

RECOMMENDED TEXTBOOKS/REFERENCES

- Mall Rajib, Fundamentals of Software Engineering, PHI
- **Software Engineering: A Practitioner's Approach**, 9th Edition, By Roger Pressman and Bruce Maxim, © 2020
- LECTURE NOTES ON SOFTWARE ENGINEERING by Dr Anayo C.I @ 2025
- LECTURE NOTES ON SOFTWARE ENGINEERING by Dr. H.S.Behera et al.

SOFTWARE ENGINEERING CONCEPTS AND PRINCIPLES

DESIGN, DEVELOPMENT, AND TESTING OF SOFTWARE SYSTEMS

**DAY 1
&
DAY 2**

What is Software vs Software Engineering?

Software (S/W) is a program or set of programs containing instructions that provide the desired functionality. Software is more than just a program code. A program is an executable code which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentation.

Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

Software Engineering?

So, we can define software engineering as an engineering branch associated with the development of software products using well-defined scientific principles, methods and procedures. The outcome of software engineering is an efficient and reliable software product.

Software Engineering (SWE) is the process of designing, developing, testing, and maintaining software.

IEEE defines software engineering as: The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.

We can alternatively view it as a systematic collection of experience. The experience is arranged in the form of methodologies and guidelines.

Software Engineering contd...

- Includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
- Rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
- By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
- Mainly used for large projects based on software systems rather than single programs or applications.
- The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
- Ensures that the software that has to be built is consistent, correct, also on budget, on time, and within the required requirements.

Concepts/Main Attributes/Need of Software Engineering

- ❖ **Efficiency:** Measure of the resource requirement of a software product.
- ❖ **Reliability:** Ensure the product will deliver the same results when used in a similar working environment.
- ❖ **Reusability:** Makes sure that the module can be used in multiple applications.
- ❖ **Maintainability:** The software is to be modified, repaired, or enhanced easily with changing requirements.
- ❖ **Large software** – Easier to build a wall than to build a house or building; likewise, as the size of software becomes large, engineering has to step in to give it a scientific process.
- ❖ **Scalability-** Software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- ❖ **Cost-** As the hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But the cost of software remains high if the proper process is not adopted.
- ❖ **Dynamic Nature-** Always-growing and adapting nature of software hugely depends upon the environment in which the user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- ❖ **Quality Management-** A Better process of software development provides better quality software products.

Characteristics of Good Software

A software product can be judged by what it offers and how well it can be used. This software must satisfy the conditions:

- ❖ Operational
- ❖ Transitional
- ❖ Maintenance

Dual Role of Software

1. As a Product

- ❖ It delivers computing potential across networks of Hardware.
- ❖ It enables the Hardware to deliver the expected functionality.
- ❖ It acts as an information transformer because it produces, manages, acquires, modifies, displays, or transmits information.

2. As a Vehicle for Delivering a Product

- ❖ It provides system functionality (e.g., payroll system).
- ❖ It controls other software (e.g., an operating system).
- ❖ It helps build other software (e.g., software tools).

Objectives of Software Engineering

- **Maintainability:** It should be feasible for the software to evolve to meet changing requirements.
- **Efficiency:** The software should not make wasteful use of computing devices such as memory, processor cycles, etc.
- **Correctness:** A software product is correct if the different requirements specified in the [SRS Document](#) have been correctly implemented.
- **Reusability:** A software product has good reusability if the different modules of the product can easily be reused to develop new products.
- **Testability:** Here software facilitates both the establishment of test criteria and the evaluation of the software concerning those criteria.
- **Reliability:** It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.
- **Portability:** In this case, the software can be transferred from one computer system or environment to another.
- **Adaptability:** In this case, the software allows differing system constraints and the user needs to be satisfied by making changes to the software.
- **Interoperability:** Capability of 2 or more functional units to process data cooperatively.

What Careers Are There in Software Engineering?

- Following are the job choices in software engineering:
- SWE (Software Engineer)
- SDE (Software Development Engineer)
- Web Developer
- Quality Assurance Engineer
- Web Designer
- Software Test Engineer
- Cloud Engineer .
- Front-End Developer
- Back-End Developer
- DevOps Engineer.
- Security Engineer.

What Tasks do Software Engineers do?

Requirement Analysis: Collaborating with stakeholders to understand and gather the requirements to design and develop software solutions.

Design and Development: Creating well-structured, maintainable code that meets the functional requirements and adheres to software design principles.

Testing and Debugging: Writing and conducting unit tests, integration tests, and debugging code to ensure software is reliable and bug-free.

Code Review: Participating in code reviews to improve code quality, ensure adherence to standards, and facilitate knowledge sharing among team members.

Maintenance: Updating and maintaining existing software systems, fixing bugs, and improving performance or adding new features.

Documentation: Writing clear documentation, including code comments, API documentation, and design documents to help other engineers and future developers understand the system.

What Skills do Software Engineers Need?

Technical skills:

- ❖ Coding and computer programming
- ❖ Software testing
- ❖ Object-oriented design (OOD)
- ❖ Software development

Soft skills:

- Communication skills
- Team player
- Problem solving
- Attention to detail

Program vs Software Product

Parameters	Program	Software Product
Definition	A program is a set of instructions that are given to a computer in order to achieve a specific task.	Software is when a program is made available for commercial business and is properly documented along with its licensing. Software Product = Program + Documentation + Licensing.
Stages Involved	Program is one of the stages involved in the development of the software.	Software Development usually follows a life cycle, which involves the feasibility study of the project, requirement gathering, development of a prototype, system design, coding, and testing.

Questions For Practice

1. A software configuration management tool helps in:

- (A) keeping track of the schedule based on the milestone reached
- (B) maintaining different versions of the configurable items
- (C) managing manpower distribution by changing the project structure
- (D) all of the above

2. Which of the following statements is/are true?

P: Software Reengineering is preferable for software products having high failure rates, poor design, and/or poor code structure.

Q: Software Reverse Engineering is the process of analyzing software with the objective of recovering its design and requirement specification.

- (A) P only
- (B) Neither P nor Q
- (C) Q only
- (D) Both P and Q

3. The diagram that helps in understanding and representing user requirements for a software project using UML (Unified Modeling Language) is:

- (A) Entity Relationship Diagram
- (B) Deployment Diagram
- (C) Data Flow Diagram
- (D) Use Case Diagram

Software Design, development, & testing of software systems

Day 2

ASSIGNMENT (Career-Oriented and Practice-Based)

INSTRUCTIONS

- Choose one of the topics/titles, and develop a mini term paper not more than 11 pages. Include Preliminary pages, such as cover page, dedication, acknowledgement, and table of contents. Scale: preliminary pages (4 pages), main body (6 pages), and reference (1 page). Submission deadline: **Last Monday of January 2026 by 9 am. You MUST attach a Plagiarism report and an AI-generated score report page while submitting.** Note: (i) A maximum of two persons per topic, (ii) A4 paper, (iii) Single Line Spacing, (iv) Font: Times New Roman, Font size: 12

- https://docs.google.com/document/d/1Zp-Zz1Bv1_AZEyalo2XCdn87E2hS5DgpPwf6nVQqiK0/edit?usp=sharing

Fundamental Principles

- **DRY (Don't Repeat Yourself):** Avoid duplication of code or logic.
- **KISS (Keep It Simple, Stupid):** Design systems as simply as possible.
- **YAGNI (You Aren't Gonna Need It):** Don't add features until they're necessary.
- **Separation of Concerns:** Divide the system into distinct sections, each handling a specific concern.
- **Continuous Improvement:** Regularly refactor and optimise code for performance and clarity.

Software Design

Software design is the **process of defining the architecture, components, interfaces, and data** of a system to meet specified requirements.



Levels of Design

High-Level Design (Architectural Design):

- Defines overall system structure and interaction between components.
- Uses *UML diagrams*, *architecture patterns* (e.g., layered, client-server, microservices).
- Example: MVC (Model-View-Controller) architecture in web applications.

Low-Level Design (Detailed Design)

- Specifies algorithms, data structures, database schema, and component logic.
- Converts high-level modules into implementable units.

Design Principles

- **Cohesion:** Components should perform a single, well-defined function.
- **Coupling:** Minimise dependencies between modules.
- **Information Hiding:** Limit the scope of data to where it's needed.
- **Design Patterns:** Common solutions to recurring problems (e.g., Singleton, Factory, Observer).

Design Tools

- UML Diagrams:** Use Case, Class, Sequence, Activity, Component, Deployment.
- CASE Tools:** Rational Rose, StarUML, Visual Paradigm, Enterprise Architect.

Software Development

Software development involves **coding, integrating, and implementing** the system based on design specifications.

Programming Concepts

- **Structured Programming:** Use of control structures — sequence, selection, and iteration.
- **Object-Oriented Programming (OOP):** Focuses on classes, inheritance, and polymorphism.
- **Functional Programming:** Uses pure functions and immutability.
- **Event-Driven Programming:** Executes logic based on events (e.g., UI clicks, signals).

Development Best Practices

- Version control (e.g., Git, GitHub).
- Code documentation and commenting.
- Modular and clean coding.
- Use of integrated development environments (IDEs).
- Continuous integration/testing pipelines.

Software Testing

Software testing is the **process of evaluating a system** to ensure it

Objectives

- ❖ Verify correctness and completeness.
- ❖ Validate system behavior against specifications.
- ❖ Detect and fix bugs early.
- ❖ Ensure reliability and performance.

Types of Testing

A. Functional Testing

- **Unit Testing:** Testing individual modules (e.g., functions, classes).
- **Integration Testing:** Ensuring components work together correctly.
- **System Testing:** Validating the complete system against requirements.
- **Acceptance Testing:** Confirming the system meets user needs (UAT).

B. Non-Functional Testing

- **Performance Testing:** Load, stress, and scalability tests.
- **Security Testing:** Checking for vulnerabilities (e.g., SQL injection, authentication flaws).
- **Usability Testing:** Evaluating user interface and experience.
- **Compatibility Testing:** Ensuring software works across platforms/devices.
-

Integration of Design, Development, and Testing

These phases are interdependent and iterative:

- **Design** informs **development** — code must reflect the architecture.
- **Testing** feeds back into both — defects lead to design or code modifications.
- In Agile and DevOps, these processes happen **continuously**, ensuring rapid improvement and quality assurance.

Documentation and Maintenance

Documentation: Crucial for system understanding, onboarding, and maintenance. Includes requirements docs, design docs, user manuals, and test reports.

Maintenance Types:

- **Corrective:** Fixing defects.
- **Adaptive:** Modifying the system for new environments.
- **Perfective:** Improving performance or maintainability.
- **Preventive:** Refactoring to avoid future issues.



Conclusion

- Design, development, and testing form the **core of software engineering practice**. Adherence to principles such as **modularity, abstraction, reusability, and continuous improvement** ensures the creation of reliable, maintainable, and scalable systems. Modern methodologies like **Agile** and **DevOps** integrate these processes seamlessly, fostering collaboration, faster delivery, and higher software quality.



Discussion

- Software engineering is essential for the functioning of government, society, and national and international businesses and institutions. Discuss?
- **Why do Software Engineers require technical and Soft Skills?**
- **Discuss Software Concepts and Software Principles known to you**

DAY 3

INTRODUCTION TO THE SOFTWARE LIFE CYCLE

AN OVERVIEW

-
- **Software Development Life Cycle (SDLC)** is a structured process that is used to design, develop, and test high-quality software. SDLC is a methodology that defines the entire procedure of software development step-by-step.
 - The goal is to deliver high-quality, maintainable software that meets the user's requirements.
 - **SDLC in software engineering models** outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users' requirements.

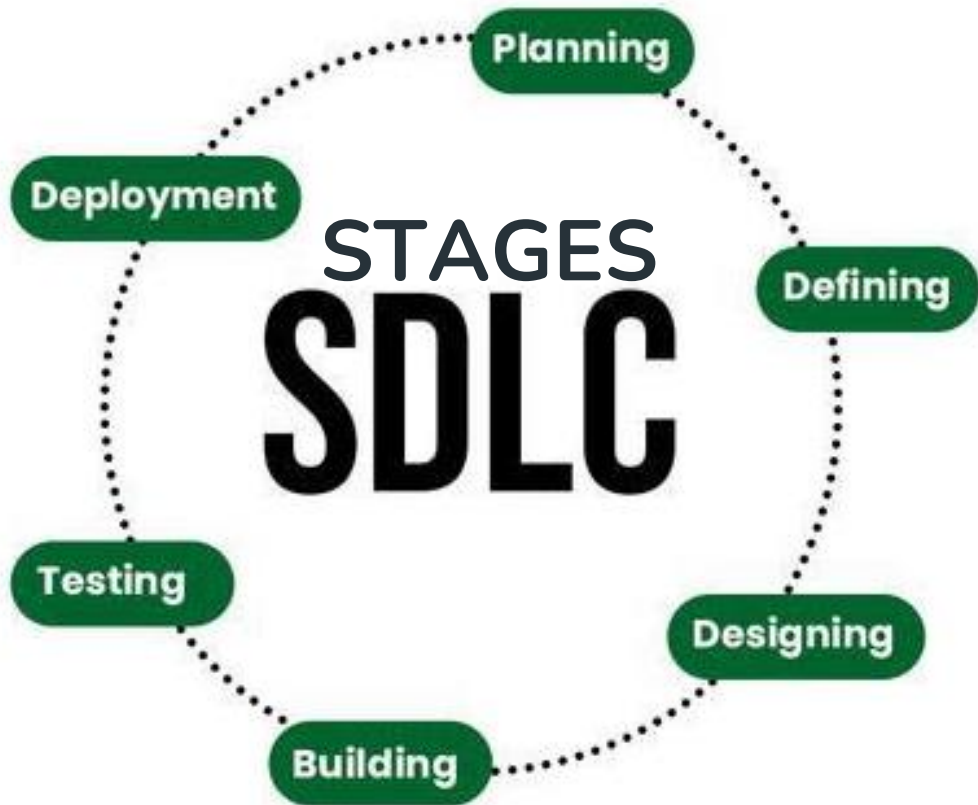
PHASES OF THE SOFTWARE LIFE CYCLE

- 1. Requirement Analysis
- 2. System Design
- 3. Implementation (Coding)
- 4. Testing
- 5. Deployment
- 6. Maintenance
- 7. Retirement

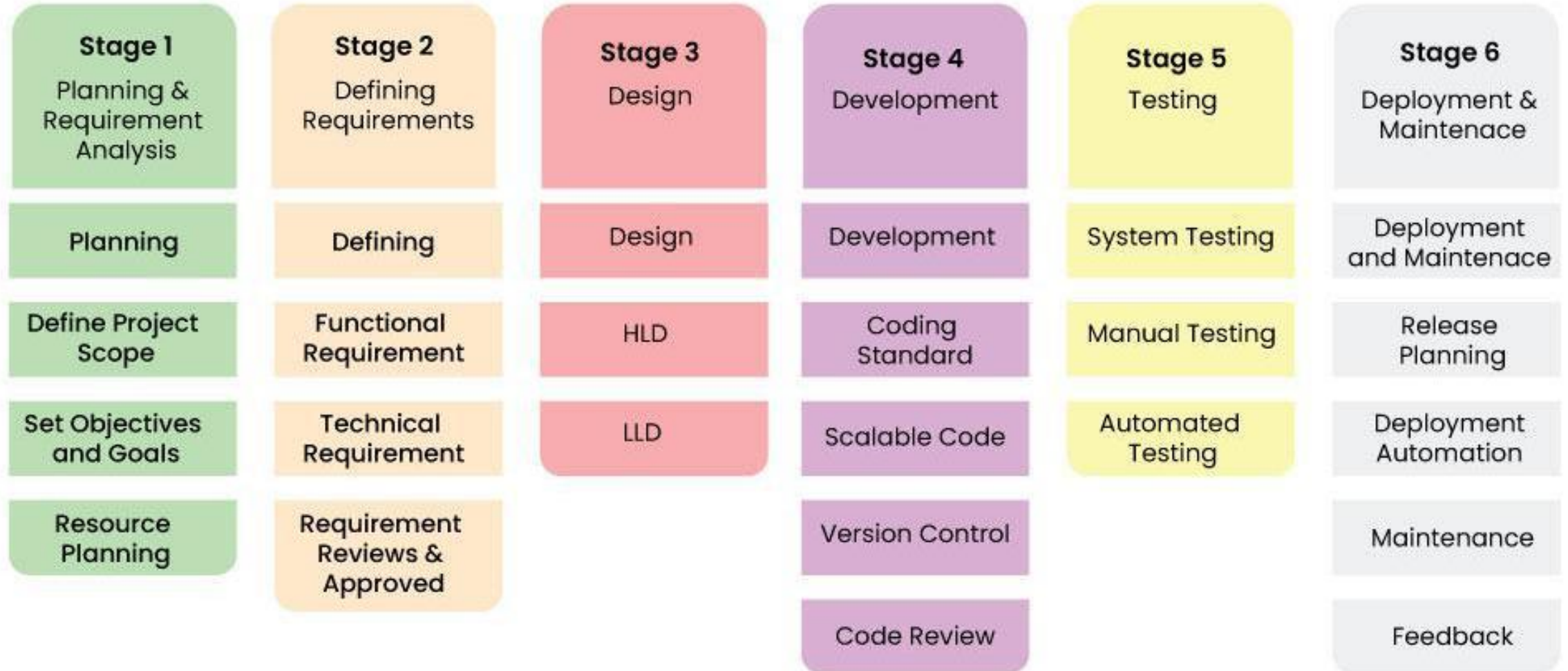
Core Difference: Scope and Coverage

Aspect	Software Life Cycle (SLC)	Software Development Life Cycle (SDLC)
Definition	The <i>entire lifespan</i> of a software product — from initial idea or conception through development, use, maintenance, and retirement.	A structured process or subset of the SLC that covers only the <i>development phase</i> — from requirement gathering to deployment.
Scope	Broader — includes post-deployment activities (maintenance, upgrades, and decommissioning).	Narrower — focuses on building and delivering the software product.
Focus	Manages software evolution over time (creation → operation → retirement).	Manages development workflow and engineering processes.
Duration	Lasts for the entire lifetime of the software.	Ends when the software is deployed or released to users.
Example Activities	Idea generation, feasibility study, development, operation, maintenance, disposal.	Requirements analysis, design, coding, testing, deployment.

WHAT IS THE SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)?



- SDLC is a process followed for software building within a software organisation.
- SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software.
- The life cycle defines a method for improving the quality of software and the overall development process.



6 Stages of Software Development Life Cycle



CASE SCENARIO: UNIVERSITY COURSE REGISTRATION SYSTEM

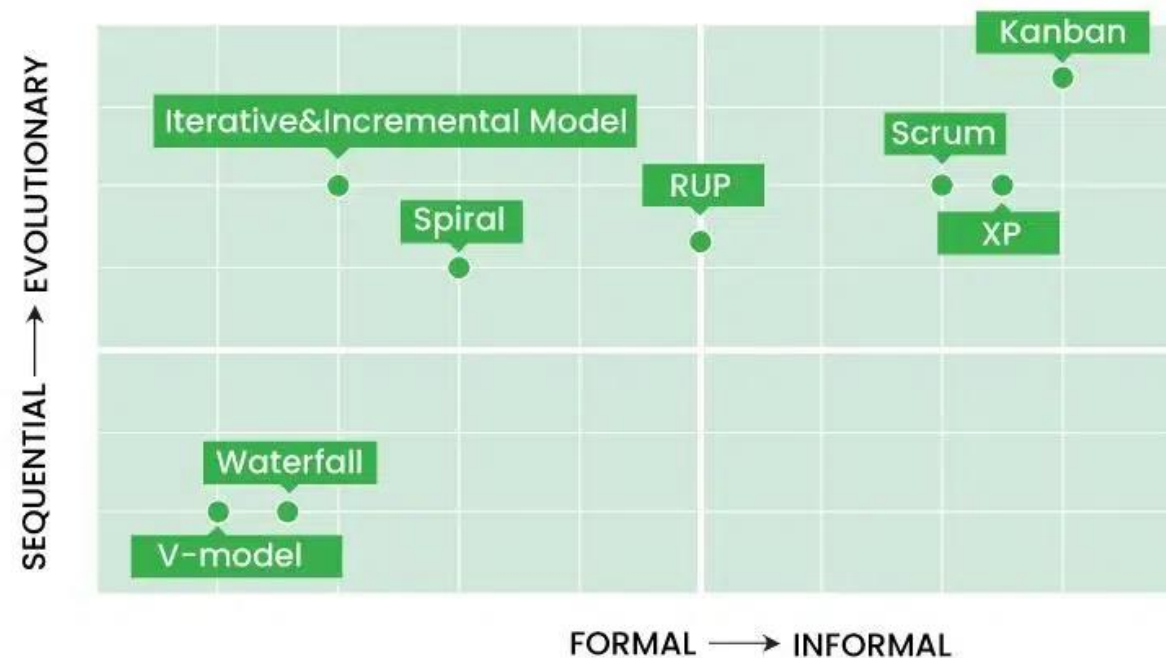
1. Requirements: Students register and pay online.
2. Design: 3-tier architecture (Web, Server, Database).
3. Implementation: Built using Django + MySQL.
4. Testing: Integration tests for payment module.
5. Deployment: Hosted on university server.
6. Maintenance: Feature updates and bug fixes.

SOFTWARE DEVELOPMENT LIFE CYCLE MODELS

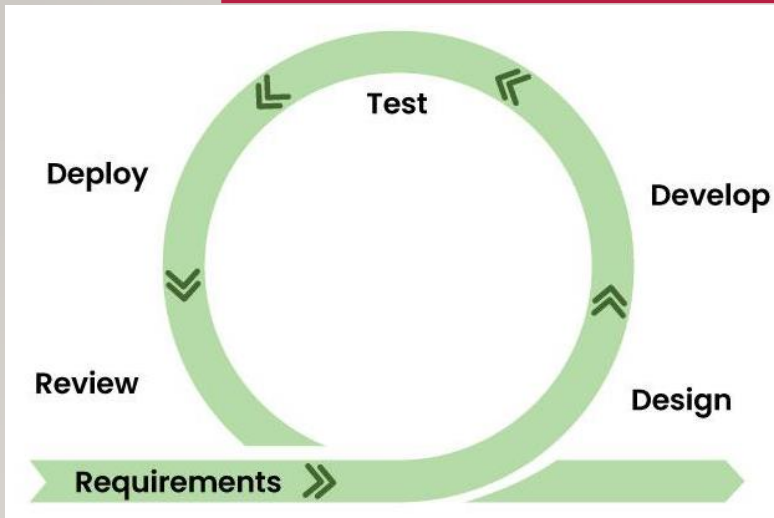
Software Development Models are structured frameworks that guide the planning, execution, and delivery of software projects. They define the **sequence of development stages**, such as requirements, design, coding, testing, and deployment.

- **Common Models**

- [Waterfall Model](#)
- [Agile Model](#)
- [V-Model](#)
- [Spiral Model](#)
- [Incremental Model](#)
- [RAD Model](#)



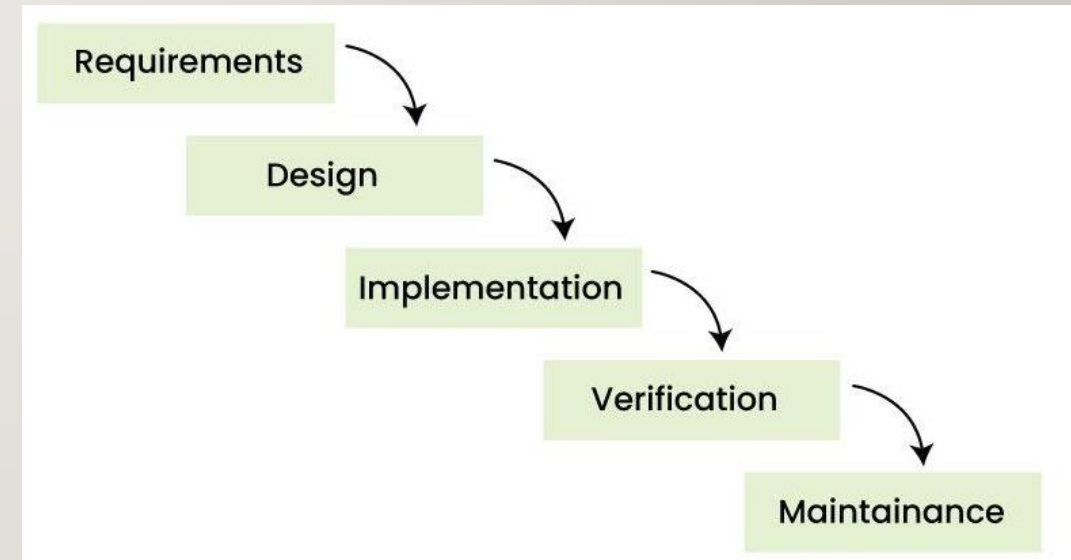
AGILE DEVELOPMENT MODEL



- It is a combination of iterative and incremental models, that is, it is made up of iterative and incremental models.
- In the Agile model, focus is given to process adaptability and customer satisfaction.
- In earlier times, the iterative waterfall model was used to create software. But in today's time, developers have to face many problems.
- The biggest problem is that in the middle of software development, the customer asks to make changes to the software. It takes a lot of time and money to make these changes.

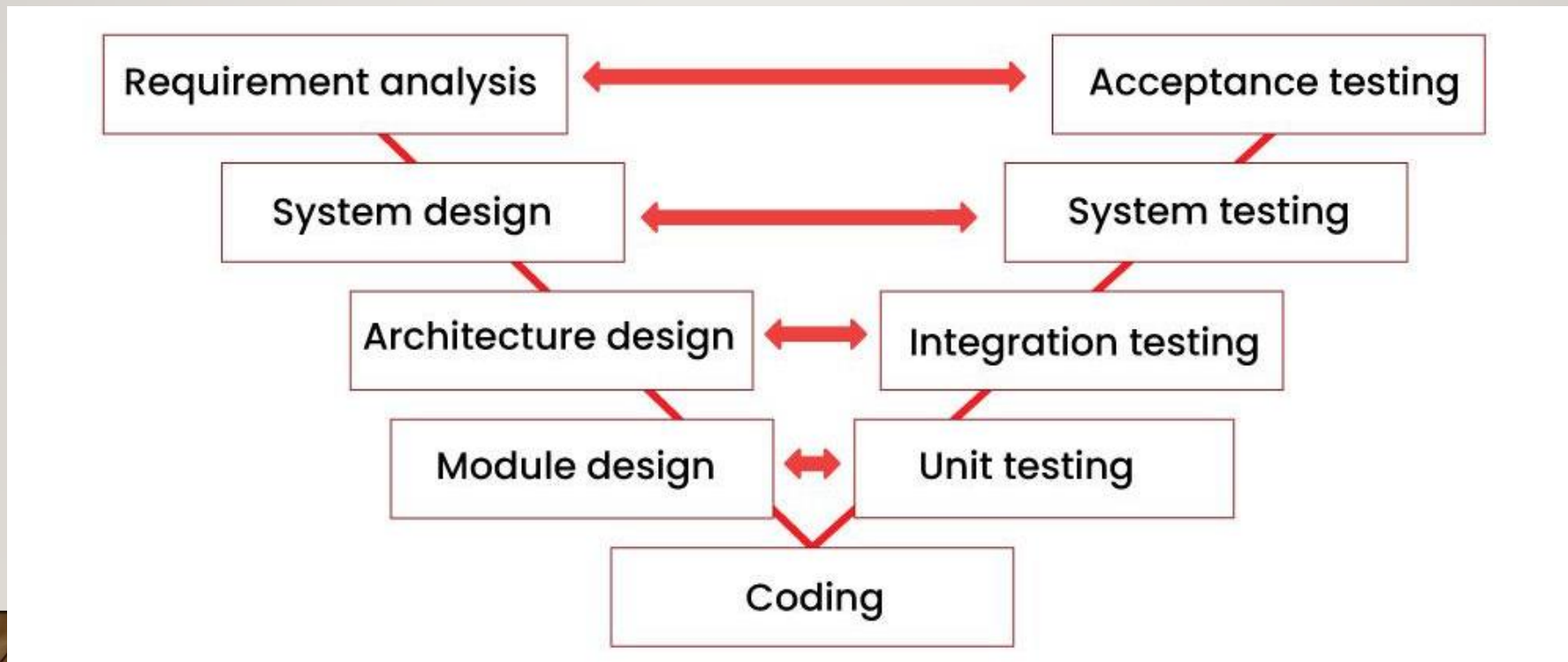
WATERFALL MODEL

- The Waterfall Model is a famous and good version of SDLC for software engineering.
- The waterfall model is a linear and sequential model, which means that a development phase cannot begin until the previous phase is completed.
- We cannot overlap phases in the waterfall model.



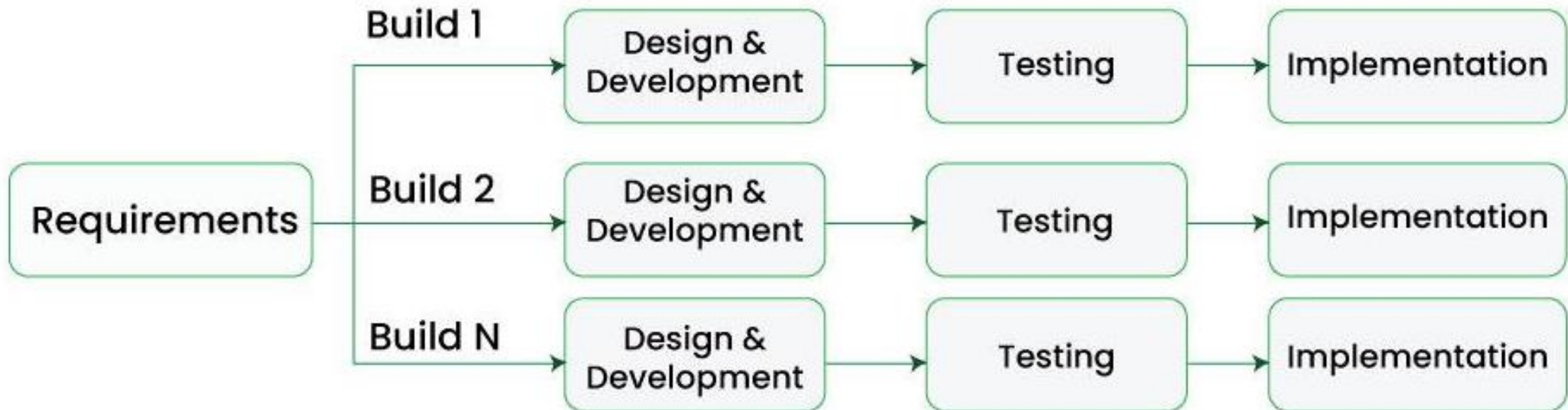
V-MODEL

- V-Model is an SDLC model, also called the Verification and Validation Model. V-Model is widely used in the **Software Development Process**, and it is considered a disciplined model. In the V-Model, the execution of each process is sequential; that is, the new phase starts only after the previous phase ends.



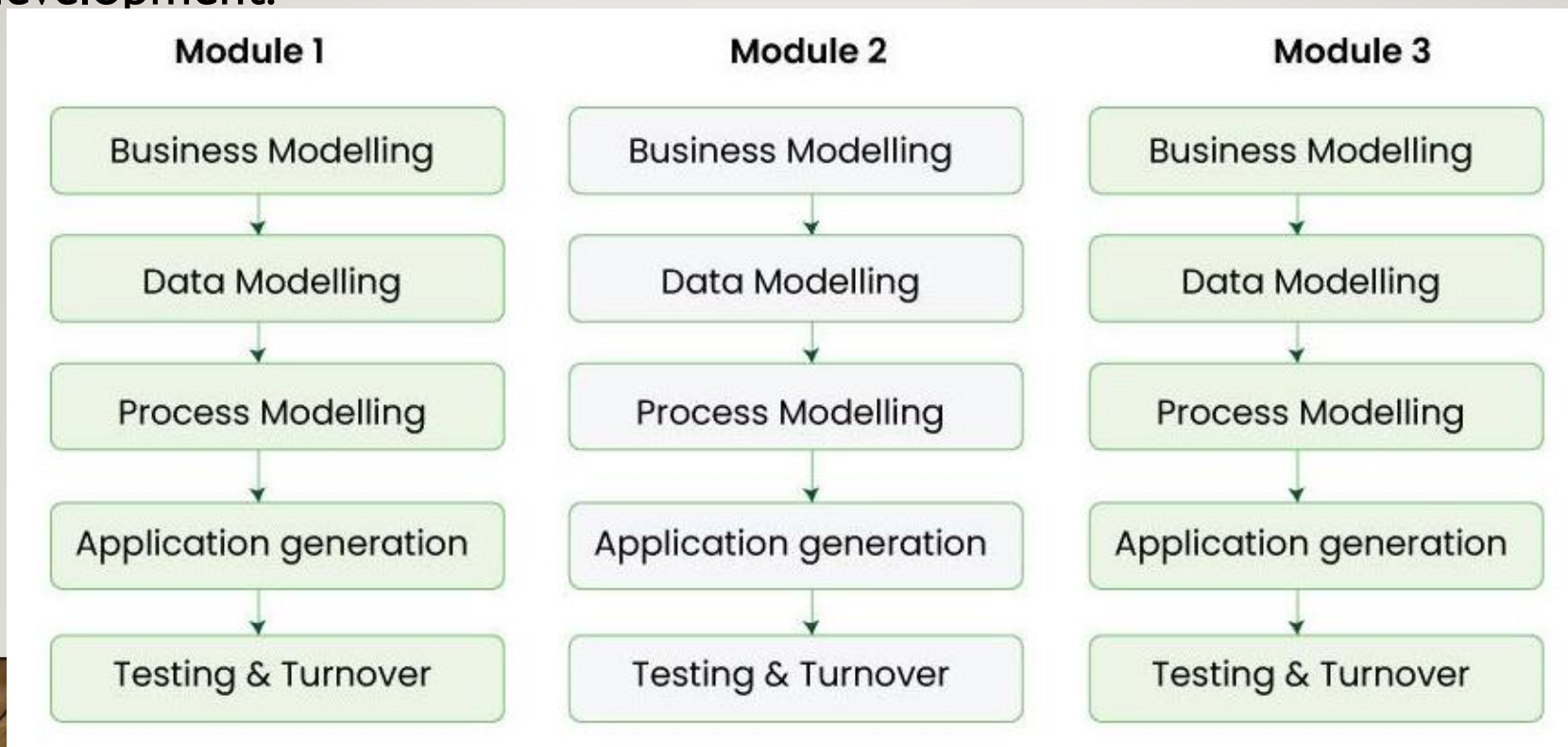
INCREMENTAL MODEL

- In the Incremental Model, the Software Development Process is divided into several increments and the same phases are followed in each increment. In simple language, under this model, a complex project is developed in many modules or builds.



RAD MODEL

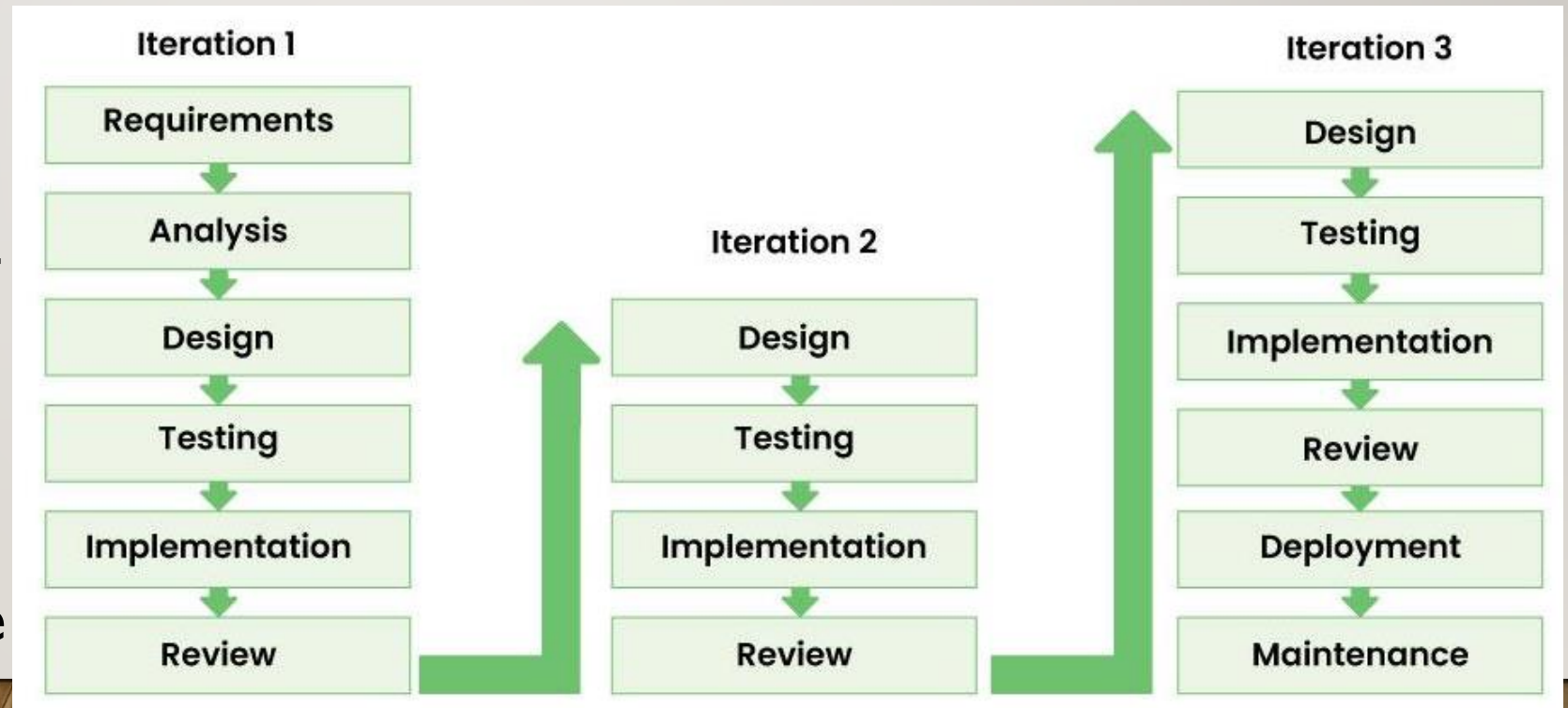
- The RAD Model stands for rapid application development model. The methodology of the RAD model ~~is similar to that of the incremental or waterfall model. It is used for small projects.~~
- The main objective of the RAD model is to reuse code, components, tools, and processes in project development.



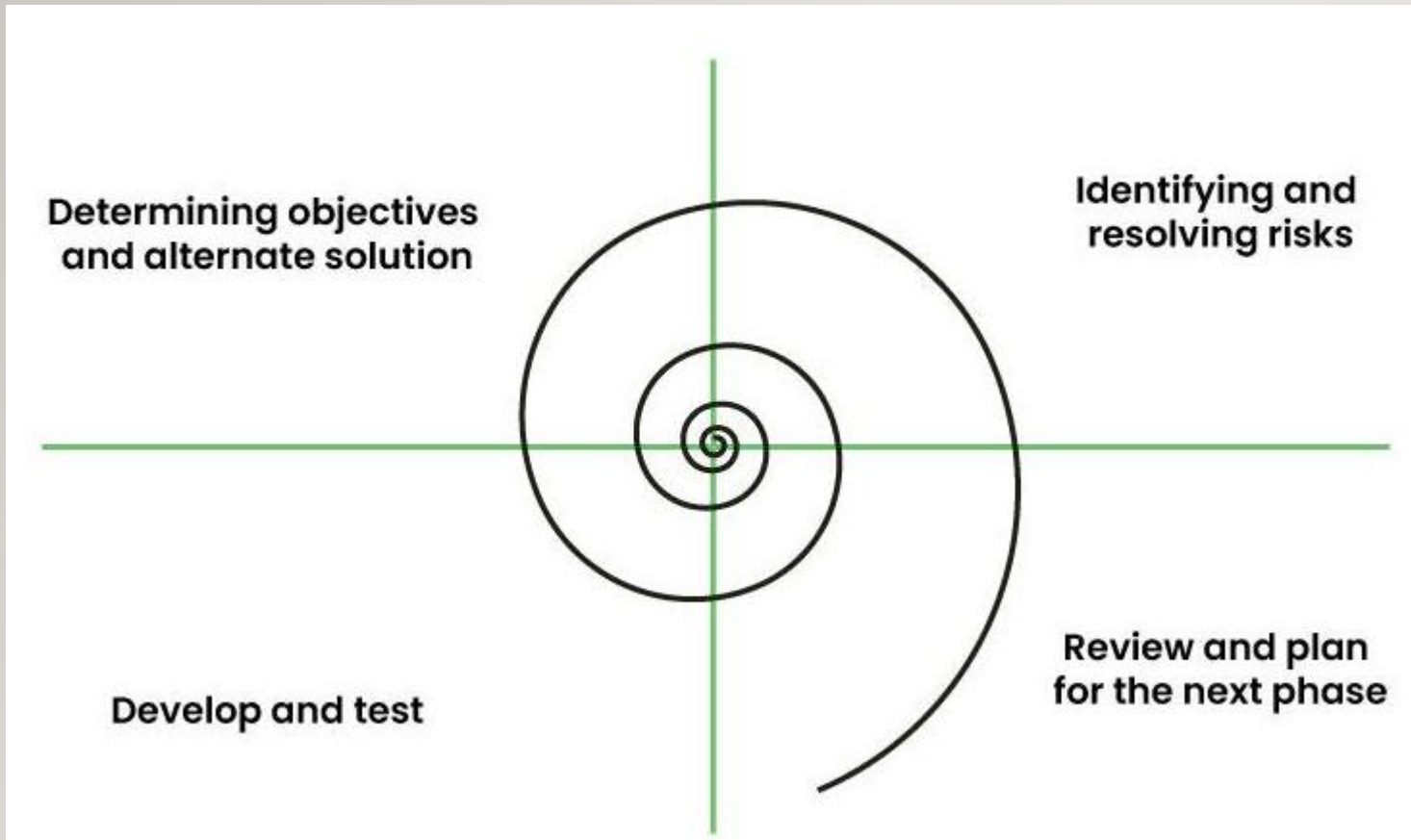
ITERATIVE MODEL

11/9/2025

- In the Iterative Model, we start developing the software with some requirements, and when it is developed, it is reviewed.
- If there are requirements for changes in it, then we develop a new version of the software based on those requirements.
- This process repeats itself many times until we get our final product.



SPIRAL MODEL

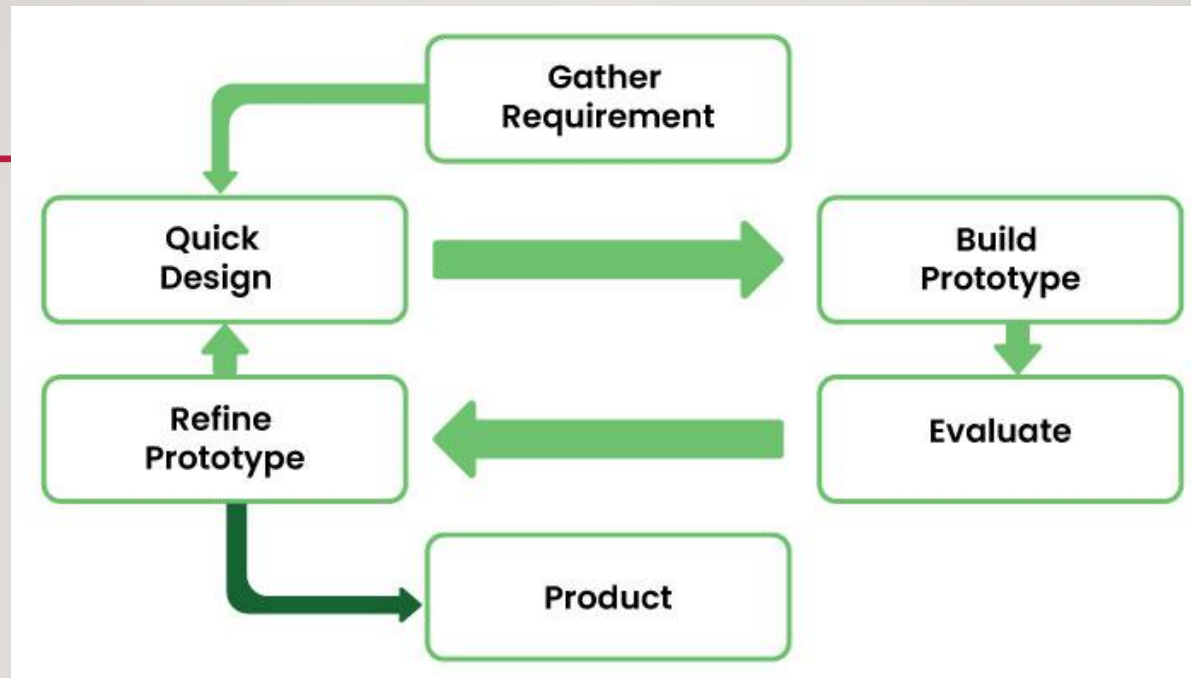


- The Spiral Model is a software development process model. This model has characteristics of both iterative and waterfall models. This model is used in projects which are large and complex.

This model was named spiral because if we look at its figure, it looks like a spiral, in which a long, curved line starts from the centre point and makes many loops around it.

The number of loops in the spiral is not decided in advance, but it depends on the size of the project and the changing requirements of the user.

PROTOTYPE MODEL



- Prototype Model is an activity in which prototypes of software applications are created. First, a prototype is created, and then the final product is manufactured based on that prototype.
- One problem in this model is that if the end users are not satisfied with the prototype model, then a new prototype model is created again, because this model consumes a lot of money and time.

WHY COMPANIES ARE SHIFTING TOWARD AGILE SOFTWARE DEVELOPMENT MODELS?

- Earlier, the ~~iterative waterfall model was used to create software. But in today's time,~~ developers have to face many problems.
- The biggest problem is that in the middle of software development, the customer asks to make changes to the software. It takes a lot of time and money to make these changes.
- The agile model was created mainly to make changes in the middle of Software Development so that the software project can be completed quickly.
- Agile focuses on a customer-centric approach that constantly takes feedback from customers and makes changes in the middle of the software development process according to their needs and expectations.
- Agile Methodologies like Scrum adopt an iterative and incremental approach, which accelerates the time-to-market for new features and products.

CONCLUSION

- Choosing the right software development model is important for the Success of a Project. Each model offers unique advantages and is suitable for Different Types of Projects.
- Agile Methodologies, due to their flexibility and customer-centric approach, are increasingly popular in the industry.
- By adopting Agile, companies can accelerate time-to-market and better adapt to changing requirements, leading to higher customer satisfaction and Successful Project Outcomes.

SUMMARY AND KEY TAKEAWAYS

- Software Life Cycle = Blueprint of software evolution.
- Each phase has distinct goals and outputs.
- Choice of model depends on project context.
- Modern SDLCs focus on agility, iteration, and quality.

DAY 4

Software Requirements, design, and testing

Requirement-Based Testing in Software Development

- The requirement-based testing approach aligns closely with the defined requirements of the software, systematically verifying that each element functions as expected. It also examines the software against predetermined criteria that help to identify any deviations or shortcomings.

- **What is Requirement-Based Testing in Software Development**
 - Requirement-based testing in software development refers to a crucial process that involves validating a software system based on its specified criteria.
 - This approach guarantees that the software aligns with documented specifications and meets the anticipated outcomes outlined during the initial phases of the project.
 - Testers generate test cases based on these requirements, methodically assessing each element to ensure proper functionality. Making testing strategies align with the requirements enhances the chances of delivering a software product that complies with predefined benchmarks and user expectations.

Principles of Requirement-Based Testing

- **Traceability:** The approach aims to establish clear links between each test and its respective requirements, ensuring easy tracking.
- **Early Engagement:** Early involvement in testing allows teams to comprehend, validate, and clarify requirements, minimising misinterpretation risks.
- **Validation and Verification:** This methodology focuses on both aspects to ensure software compliance with specified requirements, boosting testing reliability.
- **Adaptability:** Flexibility is a crucial aspect, enabling adjustments in requirements without compromising testing objectives.
- **Coverage:** Comprehensive testing coverage ensures systematic testing of all identified requirements throughout the testing phase.
- **Documentation:** Accurate documentation of requirements, test cases and results captures the entire testing process for reference.



Significance of Requirement Based Testing in Software Development

1. Alignment with Design Specifications

This testing approach ensures that the software aligns precisely with the functionalities and specifications outlined during the initial design phase. By testing against these specifications, it validates that the software behaves as intended.

2. Verification of Documented Requirements

This process ensures that the developed software accurately reflects the documented requirements. It's a critical step in ensuring that the software is constructed according to the defined criteria and expected standards.

3. Systematic Feature Evaluation

Through Requirement-Based Testing, software teams methodically evaluate each implemented feature against predetermined criteria. This thorough assessment ensures that every element functions as intended and meets the specified requirements.

4. Inconsistency Identification

This testing method plays a crucial role in detecting any disparities or deviations from the specified requirements. Recognizing these discrepancies at an early development stage enables teams to tackle and rectify them, before they escalate into significant issues.

5. Improvement in the reliability and overall quality

Requirement testing plays a major role in enhancement of the overall quality, features and dependability of the software. By ensuring that documented requirements are aligned, it boosts the software's functionality, performance and user satisfaction, leading to a more reliable end product.

Types of Requirements in Software Development

- **Functional Requirements:** They detail specific tasks, calculations, data handling and built-in features.
- **Non-Functional Requirements:** They cover attributes like performance, speed, user-friendliness, security and reliability.
- **Business Requirements:** They define the primary goals and desired outcomes, making sure the software serves a higher purpose.
- **User Requirements:** They consider user preferences, usability criteria, and accessibility concerns.
- **System Requirements:** These define the essential environment and infrastructure the software needs to run smoothly. They specify hardware compatibility, software dependencies, network configurations, and other technical parameters.
- **Stakeholder Requirements:** These gather valuable input from all parties involved in the software's development and use, including customers, users, developers, and even regulatory entities. They ensure diverse perspectives are heard and addressed.
- **Interface Requirements:** These define how the software interacts with external systems and platforms. They cover communication protocols, data formats, and API specifications.



Conclusion

- Requirement-Based Testing offers a structured approach to validate software against specified requirements.
- The advantages include enhanced product quality, early issue identification and improved traceability, contributing significantly. It also ensures software reliability and meets user expectations.
- Therefore, forming a fundamental aspect of the software development lifecycle, its effectiveness significantly relies on a clear understanding of requirements and an adaptable approach to meet them.



Types of software testing

► Unit testing

Unit testing focuses on evaluating individual units or components of a software application. This process helps ensure each part functions correctly. These tests are designed to check the smallest parts of an application, focusing on individual methods and functions. One of the key benefits of unit testing is the ability to find and fix fundamental issues quickly. Unit tests are generally inexpensive to automate and run quickly, making them a crucial part of the development process.

► Integration testing

Integration testing is the process of testing interactions between integrated components to identify any issues. Its purpose is to combine previously unit-tested components and verify that they function correctly together. This type of testing helps discover and secure interface defects that may arise when different modules interact.



Types of software testing

► System testing

System testing evaluates the fully integrated system's compliance with specified requirements. This phase assesses how well the complete integrated system meets its specified requirements and verifies that all features, functions, and requirements perform as expected. System testing can detect bugs that were missed during earlier testing levels.

► User acceptance testing

User acceptance testing is the final level of software testing to ensure product readiness for delivery. This phase assesses whether the product meets business criteria and performs the desired tasks. It provides feedback from users and reduces the risk of major bugs.

► Performance testing

The purpose of performance testing is to evaluate application performance under the expected workload. It aims to identify bottlenecks and ensure the system can handle the expected user load. Performance testing measures system performance, sensitivity, reactivity, and stability under load conditions. Aspects of application performance measured include response times and behavior with significant data.

Types of software testing

► Security testing

Security testing, including penetration testing, focuses on finding loopholes and weaknesses in the system. Its purpose is to identify vulnerabilities and ensure data protection. Security testing aims to uncover vulnerabilities that could be exploited by cybercriminals, such as SQL injection and cross-site scripting.

► Regression and smoke testing

Regression and smoke testing are critical for maintaining software quality. Regression testing checks that recent changes have not adversely affected existing functionalities, while smoke testing ensures that the most critical functionalities work before proceeding with deeper testing. Automating regression tests is recommended as they are often repetitive and time-consuming.

► Regression testing

Regression testing is crucial as it ensures that the software operates correctly after any changes have been made. The need for regression testing arises from changes such as bug fixes, updates, or configuration alterations, as well as when a new module is added. Functional tests assess the entire application, while unit tests focus on individual components, ensuring comprehensive coverage.

► Smoke testing

Smoke testing acts as a quick assessment of essential features to determine if a build is stable enough for further testing. The primary aim is to ensure that critical functionalities are working as intended before proceeding with detailed testing.



Key takeaways

- The Software Testing Life Cycle (STLC) includes key stages such as requirements analysis, test planning, test design, execution, and maintenance, which collectively enhance product quality and facilitate quick bug resolution.
- Effective software testing not only reduces costs by identifying defects early in the development process but also ensures reliability through various testing types, including manual, automated, functional, and non-functional testing.
- Implementing best practices like early and continuous testing contributes to higher software quality, minimizes the risk of significant issues, and builds stakeholder confidence in the reliability of the final product.

